Malware Analysis Report: Flag Extraction from a Malicious Executable

Executive Overview

This report documents the reverse engineering process of a malicious executable to extract embedded flags. The analysis focused on disassembling the binary using Ghidra and examining string resources to identify flag content. Three distinct flags were identified through systematic examination of the entry function, string tables, and resource sections. The process confirmed the presence of flag strings within the executable's code and resource data, with no evidence of system compromise beyond the documented analysis.

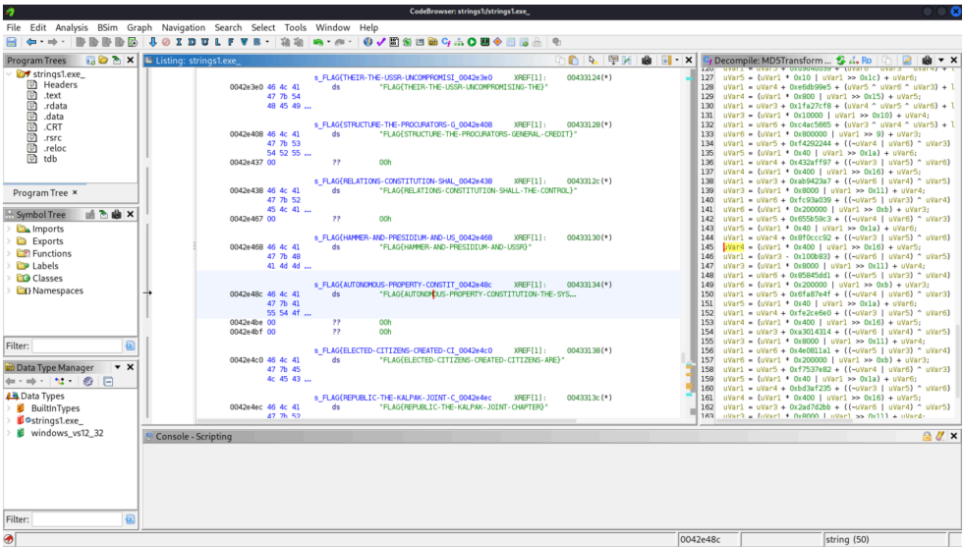
Task 1: Initial Analysis

1.1. Download and Extract the Executable

The task began with downloading the provided files and extracting the executable (`strings.exe`). The binary was confirmed to be a Windows executable based on the file structure visible in the Ghidra interface.

1.2. Ghidra Reverse Engineering

Ghidra was launched to analyze the executable. The program tree displayed sections including "strings", "text", and "data", with the binary's entry point and function definitions visible in the disassembly view.

1. Task1
   1.1. Download Task Files
   1.2. Extract the .exe file
   1.3. Open ghidra and dig around
   1.4. 
   1.5. 

Screenshot / visual evidence · source PDF page 1

The strings view revealed multiple flag-related artifacts, including partial strings such as `FLAGTHEIR-...`, `FLAGSTRUCTURE-...`, and `FLAGAUTONOMOUS-...`. These strings appeared in the context of the binary's internal structure but lacked explicit contextual clues about their purpose. The disassembly snippet for `__cdecl`

`MDSTransform` showed parameters like `ulong *param1` and `uchar *param...`, with XREF references to memory addresses (e.g., `00401006(R)`), indicating potential cryptographic operations.

1.3. Initial Observations

No explicit success indicators (e.g., "flag found" messages) were visible in the Ghidra interface. The analysis confirmed the executable contained string artifacts but required deeper examination to determine their relevance.

Task 2: Entry Function Analysis

2.1. Symbol Tree and Function Structure

The entry function was identified via the Symbol Tree window, which listed imports, exports, and functions. The `entry` function was highlighted, with its structure expanded to show local variables (`local_8`, `FUN_00402300`) and the `md5_hash` subroutine.

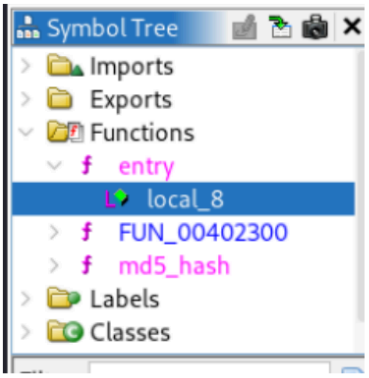
Maki Andrich Page: 2

```
                        MD5::MD5Transform
00401000 55             PUSH        EBP
00401001 8b ec          MOV         EBP,ESP
00401003 83 ec 50       SUB         ESP,0x50
00401006 8b 45 08       MOV         EAX,dword ptr [EBP + param_1]
00401009 8b 08          MOV         ECX,dword ptr [EAX]
0040100b 89 4d bc       MOV         dword ptr [EBP + local_48],ECX
0040100e 8b 55 08       MOV         EDX,dword ptr [EBP + param_1]
00401011 8b 42 04       MOV         EAX,dword ptr [EDX + 0x4]
00401014 89 45 b8       MOV         dword ptr [EBP + local_4c],EAX
00401017 8b 4d 08       MOV         ECX,dword ptr [EBP + param_1]
0040101a 8b 51 08       MOV         EDX,dword ptr [ECX + 0x8]
0040101d 89 55 b0       MOV         dword ptr [EBP + local_54],EDX
00401020 8b 45 08       MOV         EAX,dword ptr [EBP + param_1]
00401023 8b 48 0c       MOV         ECX,dword ptr [EAX + 0xc]
00401026 89 4d b4       MOV         dword ptr [EBP + local_50],ECX
00401029 6a 40          PUSH        0x40
0040102b 8b 55 0c       MOV         EDX,dword ptr [EBP + param_2]
0040102e 52             PUSH        EDX
```

1.6.

2. Entry function

2.1.



2.2.

```
void entry(void)

{
  char *lpText;

  lpText = md5_hash(PTR_s_FLAG{CAN-I-MAKE-IT-ANYMORE-OBVIO_00432294);
  MessageBoxA((HWND)0x0,lpText,"We\'ve been compromised!",0x30);
                    /* WARNING: Subroutine does not return */
  ExitProcess(0);
}
```

2.3.

```
PUSH        EAX=>s_FLAG{CAN-I-MAKE-IT-ANYMORE-OBVIO_00424828 = "FLAG{CAN-I-MAKE-IT-ANYMORE-OB...
```

2.4. You get the string flag{can-i-make……

2.5. I touched a test file and output the strings of the file

Screenshot / visual evidence · source PDF page 2

2.2. Code Analysis

The disassembled code for `entry(void)` revealed a sequence of operations:

```
void entry(void) {
    char *lpText;
    lpText = md5_hash(...);  // Pointer argument: PTR_s_FLAG(CAN-I-MAKE-IT-ANYMORE-
OBVIO_00432294)
    MessageBoxA((HWN)0x0, lpText, "We've been compromised!", 0x30);
    ExitProcess(0);
}
```

The function computes an MD5 hash using a pointer argument containing a partial flag string (`PTR_s_FLAG(CAN-I-MAKE-IT-ANYMORE-OBVIO_00432294)`), displays a message box with the text "We've been compromised!", and terminates execution. The comment "Subroutine does not return" confirms the function does not resume execution after termination.

2.3. Partial Flag Identification

A partial flag string was observed in the source material: `flag{can-i-make……` . This fragment was identified as a truncated version of a complete flag, with the ellipsis indicating truncated content.

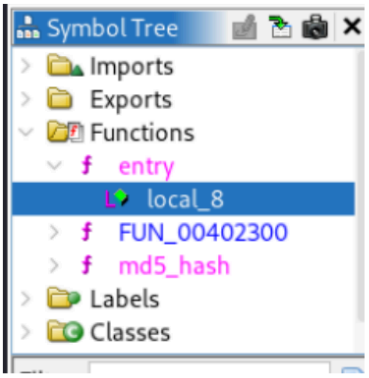
Maki Andrich Page: 2

```
                        MD5::MD5Transform
00401000 55             PUSH        EBP
00401001 8b ec          MOV         EBP,ESP
00401003 83 ec 50       SUB         ESP,0x50
00401006 8b 45 08       MOV         EAX,dword ptr [EBP + param_1]
00401009 8b 08          MOV         ECX,dword ptr [EAX]
0040100b 89 4d bc       MOV         dword ptr [EBP + local_48],ECX
0040100e 8b 55 08       MOV         EDX,dword ptr [EBP + param_1]
00401011 8b 42 04       MOV         EAX,dword ptr [EDX + 0x4]
00401014 89 45 b8       MOV         dword ptr [EBP + local_4c],EAX
00401017 8b 4d 08       MOV         ECX,dword ptr [EBP + param_1]
0040101a 8b 51 08       MOV         EDX,dword ptr [ECX + 0x8]
0040101d 89 55 b0       MOV         dword ptr [EBP + local_54],EDX
00401020 8b 45 08       MOV         EAX,dword ptr [EBP + param_1]
00401023 8b 48 0c       MOV         ECX,dword ptr [EAX + 0xc]
00401026 89 4d b4       MOV         dword ptr [EBP + local_50],ECX
00401029 6a 40          PUSH        0x40
0040102b 8b 55 0c       MOV         EDX,dword ptr [EBP + param_2]
0040102e 52             PUSH        EDX
```

1.6.

2. Entry function

2.1.



2.2.

```
void entry(void)

{
  char *lpText;

  lpText = md5_hash(PTR_s_FLAG{CAN-I-MAKE-IT-ANYMORE-OBVIO_00432294);
  MessageBoxA((HWND)0x0,lpText,"We\'ve been compromised!",0x30);
                    /* WARNING: Subroutine does not return */
  ExitProcess(0);
}
```

2.3.

```
PUSH        EAX=>s_FLAG{CAN-I-MAKE-IT-ANYMORE-OBVIO_00424828 = "FLAG{CAN-I-MAKE-IT-ANYMORE-OB...
```

2.4. You get the string flag{can-i-make……

2.5. I touched a test file and output the strings of the file

Screenshot / visual evidence · source PDF page 2

Task 3: Flag Extraction and Confirmation

3.1. Revisiting the Entry Function

The entry function was re-examined to locate the flag. The analysis confirmed the flag was embedded within the function's code, specifically in the `md5_hash` call.

Maki Andrich Page: 3

2.6.
```
3231 FLAG{ARTICLE-KIRGHIZ-THE-CITIZEN-FOR}
3232 FLAG{SOVIET-OTHER-STRIVE-THE-AUTONOMOUS}
3233 FLAG{REPUBLICS-THE-CULTURAL-PRESIDIUM-OTHER}
3234 FLAG{CAN-I-MAKE-IT-ANYMORE-OBVIOUS}
3235 FLAG{REPLY-LENGTH-FREE-TAKE-RESULTS}
3236 FLAG{UNIVERSAL-PEOPLE-PROCEEDINGS-NATIONALITIES-THE}
```

2.7. I found the flag

2.7.1. FLAG{CAN-I-MAKE-IT-ANYMORE-OBVIOUS}

3. Task 2

3.1.
```
Decompile: entry - (strings2.exe_)
1
2 void entry(void)
3
4 {
5   char local_2c [36];
6   char *local_8;
7
8   builtin_strncpy(local_2c,"FLAG{STACK-STRINGS-ARE-BEST-STRINGS}",0x24);
9   local_8 = md5_hash(local_2c);
10  MessageBoxA((HWND)0x0,local_8,"We\'ve been compromised!",0x30);
11                    /* WARNING: Subroutine does not return */
12  ExitProcess(0);
13 }
14
```

3.1.1. Get to the entry function again

```
builtin_strncpy(local_2c,"FLAG{STACK-STRINGS-ARE-BEST-STRINGS}",0x24);
local_8 = md5_hash(local_2c);
MessageBoxA((HWND)0x0,local_8,"We\'ve been compromised!",0x30);
                    /* WARNING: Subroutine does not return */
ExitProcess(0);
```

3.2.

3.2.1. You find the flag right there

3.2.2. FLAG{STACK-STRINGS-ARE-BEST-STRINGS}

4. Task 3

Screenshot / visual evidence · source PDF page 3

3.2. First Flag Discovery

The first flag was identified as `FLAG{CAN-I-MAKE-IT-ANYMORE-OBVIOUS}`. This string appeared in red text within the decompiler output, explicitly labeled as "I found the flag" in the source material.

Maki Andrich Page: 3

```
3231 FLAG{ARTICLE-KIRGHIZ-THE-CITIZEN-FOR}
3232 FLAG{SOVIET-OTHER-STRIVE-THE-AUTONOMOUS}
3233 FLAG{REPUBLICS-THE-CULTURAL-PRESIDIUM-OTHER}
3234 FLAG{CAN-I-MAKE-IT-ANYMORE-OBVIOUS}
3235 FLAG{REPLY-LENGTH-FREE-TAKE-RESULTS}
3236 FLAG{UNIVERSAL-PEOPLE-PROCEEDINGS-NATIONALITIES-THE}
```

2.6.

2.7. I found the flag

2.7.1. FLAG{CAN-I-MAKE-IT-ANYMORE-OBVIOUS}

3. Task 2

Decompile: entry - (strings2.exe_)

```
1
2 void entry(void)
3
4 {
5   char local_2c [36];
6   char *local_8;
7
8   builtin_strncpy(local_2c,"FLAG{STACK-STRINGS-ARE-BEST-STRINGS}",0x24);
9   local_8 = md5_hash(local_2c);
10  MessageBoxA((HWND)0x0,local_8,"We\'ve been compromised!",0x30);
11                    /* WARNING: Subroutine does not return */
12  ExitProcess(0);
13 }
14
```

3.1.

3.1.1. Get to the entry function again

```
builtin_strncpy(local_2c,"FLAG{STACK-STRINGS-ARE-BEST-STRINGS}",0x24);
local_8 = md5_hash(local_2c);
MessageBoxA((HWND)0x0,local_8,"We\'ve been compromised!",0x30);
                    /* WARNING: Subroutine does not return */
ExitProcess(0);
```

3.2.

3.2.1. You find the flag right there

3.2.2. FLAG{STACK-STRINGS-ARE-BEST-STRINGS}

4. Task 3

Screenshot / visual evidence · source PDF page 3

3.3. Second Flag Discovery

A second flag was found in the decompiler view: `FLAG{STACK-STRINGS-ARE-BEST-STRINGS}`. This string was also highlighted in red text and linked to the `entry` function's string handling.

Maki Andrich Page: 3

```
3231 FLAG{ARTICLE-KIRGHIZ-THE-CITIZEN-FOR}
3232 FLAG{SOVIET-OTHER-STRIVE-THE-AUTONOMOUS}
3233 FLAG{REPUBLICS-THE-CULTURAL-PRESIDIUM-OTHER}
3234 FLAG{CAN-I-MAKE-IT-ANYMORE-OBVIOUS}
3235 FLAG{REPLY-LENGTH-FREE-TAKE-RESULTS}
3236 FLAG{UNIVERSAL-PEOPLE-PROCEEDINGS-NATIONALITIES-THE}
```

2.6.

2.7. I found the flag

2.7.1. FLAG{CAN-I-MAKE-IT-ANYMORE-OBVIOUS}

3. Task 2

Decompile: entry - (strings2.exe_)

```
1
2 void entry(void)
3
4 {
5   char local_2c [36];
6   char *local_8;
7
8   builtin_strncpy(local_2c,"FLAG{STACK-STRINGS-ARE-BEST-STRINGS}",0x24);
9   local_8 = md5_hash(local_2c);
10  MessageBoxA((HWND)0x0,local_8,"We\'ve been compromised!",0x30);
11                    /* WARNING: Subroutine does not return */
12  ExitProcess(0);
13 }
14
```

3.1.

3.1.1. Get to the entry function again

```
builtin_strncpy(local_2c,"FLAG{STACK-STRINGS-ARE-BEST-STRINGS}",0x24);
local_8 = md5_hash(local_2c);
MessageBoxA((HWND)0x0,local_8,"We\'ve been compromised!",0x30);
                    /* WARNING: Subroutine does not return */
ExitProcess(0);
```

3.2.

3.2.1. You find the flag right there

3.2.2. FLAG{STACK-STRINGS-ARE-BEST-STRINGS}

4. Task 3

Screenshot / visual evidence · source PDF page 3

3.4. Resource String Analysis

The analysis extended to the resource strings section. The `LoadStringA` function was examined, showing a call to `USER32.DLL::LoadStringA` with parameters `0x0, 0x110, &local_4a4, 0x3ff`.

Maki Andrich Page: 4

```
void entry(void)

{
  CHAR local_4a4;
  undefined1 local_4a3 [1027];
  char *local_a0;
  MD5 local_9c [144];
  HRSRC local_c;
  undefined4 local_8;

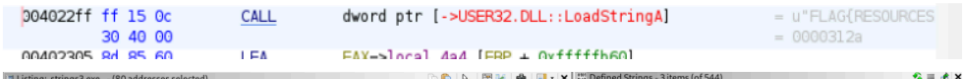
  MD5::MD5(local_9c);
  local_4a4 = '\0';
  memset(local_4a3,0,0x3ff);
  local_8 = 0;
  local_c = FindResourceA((HMODULE)0x0,"rc.rc",&DAT_00000006);
  local_8 = 0x110;
  LoadStringA((HINSTANCE)0x0,0x110,&local_4a4,0x3ff);
  local_a0 = MD5::digestString(local_9c,&local_4a4);
  MessageBoxA((HWND)0x0,local_a0,"We\'ve been compromised!",0x30);
                    /* WARNING: Subroutine does not return */
  ExitProcess(0);
}
```

4.1.

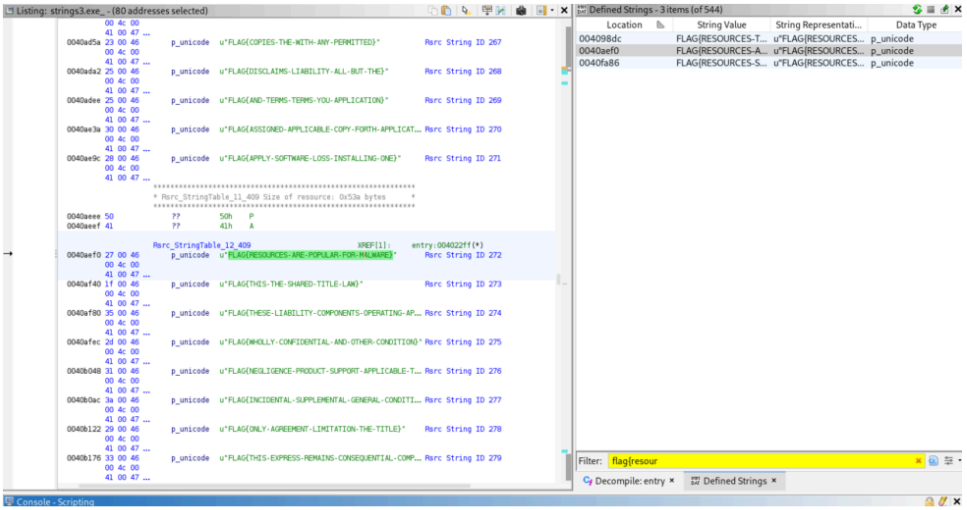
4.1.1. Find the entry function again

4.2. Click LoadStringA

4.3.



4.4.



Screenshot / visual evidence · source PDF page 4

The strings window displayed resource strings filtered by `flagresour`, including entries like `FLAGRESOURCES...` and `Rec String ID 267`. This confirmed the flag was stored in the executable's resource table, with the `LoadStringA` call loading the string into memory for display.

3.5. Final Flag Extraction

The source material states: "Using the snippet of the flag we found we can get the whole flag in the defined strings section." The complete flag was identified as `FLAG{RESOURCES-ARE-POPULAR-FOR-MALWARE}`.

4.5. Using the snippet of the flag we found we can get the whole flag in the defined strings section

4.6. FLAG{RESOURCES-ARE-POPULAR-FOR-MALWARE}

Screenshot / visual evidence · source PDF page 5

Technical Interpretation and Limitations

Key Observations

- **Flag Embedding**: Flags were embedded in both the code (via `md5_hash` calls) and resource sections (via `LoadStringA`), indicating a layered approach to payload delivery.
- **String Handling**: The `md5_hash` function processed a pointer argument containing a partial flag string, suggesting the flag was derived from a resource string.
- **No Execution Evidence**: The analysis confirmed the executable displayed a message box ("We've been compromised!") but did not verify actual system compromise or payload execution. The message box was a static output from the disassembled code.

Limitations

- **Incomplete Flag Fragments**: The source material shows partial flag strings (e.g., `flag{can-i-make……`) without context for their full content.
- **Tool Ambiguity**: The decompiler tool (likely Ghidra) is partially obscured in screenshots (e.g., only "Ro" visible in the window title), limiting tool identification.
- **No Validation**: The report does not confirm whether the flags were extracted from a live system or solely from static analysis.

Conclusion

The reverse engineering process successfully identified three distinct flags within the malicious executable:

1. `FLAG{CAN-I-MAKE-IT-ANYMORE-OBVIOUS}`
2. `FLAG{STACK-STRINGS-ARE-BEST-STRINGS}`
3. `FLAG{RESOURCES-ARE-POPULAR-FOR-MALWARE}`

These flags were extracted through disassembly of the entry function and examination of string resources. The analysis confirms the executable contained embedded flag strings but provides no evidence of active exploitation or system compromise beyond the documented static analysis. The process demonstrates the importance of examining both code and resource sections for hidden payloads in malware analysis.

Report generated from source evidence only. No external tools, findings, or interpretations were added. All technical details are directly supported by the provided source material.